IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Manuscript prepared for submission. Publication dates have not been assigned.

Digital Object Identifier Not assigned

Profile-Guided Register Allocation With a Local Potential Search and a Research Dashboard

AUTHOR NAME NOT SUPPLIED[1]
[1]Affiliation, institution, city, and country were not supplied.

Corresponding author: not supplied (e-mail: not supplied).

Funding information was not supplied. Replace this note before submission.

ABSTRACT Register allocation assigns simultaneously live values to a finite register file and accounts for the rest with memory traffic. A profile can change which value is expensive to spill, but only when every block count is supplied and the cost is defined as an estimate rather than as measured time. Graph coloring, linear scan, and learned allocators usually split ranges, coalesce copies, or emit spill code inside a production compiler. A static site count can keep a cold value, and a search that moves one range at a time can miss a joint swap. This paper documents PGARA, a batch prototype and a React research workspace for a small three-address language. The analyzer builds may-liveness and an interference graph, then runs static greedy packing, profile-weighted greedy packing, and a potential search over average and worst-profile spill estimates. The workspace shows those assignments, the graph, and a synthetic suite, and it keeps a fixed sample fixture separate from backend results. On the built-in suite, profile weighting changes one spill set from estimated cost 101 to 8, and tradeoff 1 changes the search from estimated cost 19 to 20. These figures are not application runtime. Splitting, coalescing, spill-code emission, and timed application runs remain future work.

INDEX TERMS compiler optimization, interference graph, potential game, profile-guided optimization, register allocation, spill cost

I. INTRODUCTION

REGISTER allocation is a late compiler pass. Values that remain in registers avoid the latency of memory traffic, but a target provides only finitely many physical registers. Two values that are live at the same program point cannot share a register, so the allocator assigns distinct registers or spills a value and reloads it at later uses. PGARA means profile-guided adaptive register allocation. In the system inspected here, adaptive does not mean an online controller. The caller supplies the program, the register count, and one or more execution profiles, and both the analyzer and the dashboard run as a batch.

The practical problem is that a spill decision is easy to display and hard to defend. A count of definitions and uses treats a cold block and a hot block alike. A profile can reverse that preference only when every block has an explicit count and the cost model states what a spill means. Production compilers already combine profiles with instruction selection, scheduling, and spill insertion [1], [2]. Isolating the allocation choice makes the estimate easier to audit and easier to mistake for a runtime result. The research workspace exists so that a zero estimate, a spilled range, and a sample fixture are not shown as the same kind of number.

Classical allocators color an interference graph [3]–[6] or scan live intervals in roughly linear time [7]–[9]. Pereira and Palsberg formulate the register file as a puzzle [10]. Expected distance to the next use has been used to place spill instructions [11]. Those methods are implemented in compilers that rewrite the program.

More recent work learns a policy inside LLVM [12], publishes interference graphs for learning [13], or searches with an evolutionary encoding [14]. Other papers retarget allocation to GPU register files [15], timing-speculative pipelines [16], [17], application-specific baseband kernels [18], or dual-usage vector names [19]. Their metrics are real for those systems. They do not, by themselves, say which spill set is cheaper under a caller-supplied multi-profile estimate on this intermediate representation.

The shortcomings that matter here are concrete. Coloring and scan allocators usually emit spill code and may split or coalesce ranges. PGARA does neither, so it cannot claim their code-quality results. A single spill weight hides disagreement among profiles. A search that changes one live range cannot swap two ranges in one step. Learned allocators need a training corpus and a full backend [12], [13], which this repository does not include. A dashboard that redraws one algorithm's cost as if it were the other three would hide that fact. The implementation plots each returned sum separately.

The implemented system is a pipeline together with a workspace that calls it. After parsing, the analyzer builds a control-flow graph, computes may-liveness, and adds an interference edge where two virtual registers are simultaneously live. Static greedy orders ranges by a profile-independent site count. Profile-weighted greedy orders them by the sum of estimated spill operations across the supplied profiles. The potential search starts from the all-spilled assignment by default and accepts a feasible change of one range only when the potential in Section III-H strictly decreases. FastAPI exposes health, a sample program, allocation, and a synthetic benchmark. The React workspace has six views: overview, program editor, allocation lab, interference graph, benchmark comparison, and algorithm trace.

The workspace is suitable for explanation and regression, not as a replacement allocator. Sample mode shows a fixed illustration and does not recompute when the editor changes. Backend mode sends the editor contents only when the user runs an experiment. A failed request is not replaced by the sample fixture.

The paper follows the requested outline inside the IEEE Access article format. The abstract is one paragraph within the journal's 150–250 word limit [20]. Section II discusses verified related work. Section III formulates the problem and records the architecture, the dashboard, the equations, and the algorithms taken from the source. Section IV states the tools, including the frontend commands, and the estimates that were computed. Section V states the limits and the next measurements.

II. RELATED WORK

This section discusses scholarly works whose bibliographic records were retrieved and checked. Each paragraph states the proposal, the method named by the source, the result stated in the retrieved abstract or title, and the gap relative to PGARA. Where an abstract does not give a number or an implementation tool, none is added. None of these experiments was re-run here. Their benchmarks and cost units differ from the estimated spill operations in Section III-H, so this paper does not rank the prototype against them. The brief asked for fifteen papers from the last five years, each with two paragraphs of experimental tools. Nine works from 2022–2025 were verified. The classical papers below are included because they are the methods under discussion. The shortfall is listed with the draft.

Chaitin *et al.* formulated register allocation as graph coloring [3]. The verified record is the 1981 *Computer Languages* paper. The retrieved metadata did not include an experimental abstract. A companion paper treats spilling via graph coloring [4]. Later coloring allocators build an interference graph and spill when a legal coloring is unavailable. No spill-count figure from those papers is repeated here. The gap is that coloring does not define this multi-profile estimate or the one-range search, and it does not define the workspace that displays them.

Briggs, Cooper, and Torczon added optimistic coloring and a broader treatment of rematerialization [5]. The abstract reports that optimistic coloring decreases the number of procedures that need spill code and reduces spill code when spilling remains necessary, and that rematerialization lowers the cost of spilling some values. Exact percentages were not in the retrieved abstract. PGARA does not rematerialize values and does not report procedure-level spill-code size.

George and Appel interleaved coloring reductions with conservative coalescing [6]. The method targets copy elimination. The abstract states that Briggs's conservative heuristic leaves too many moves and that iterated coalescing addresses that conservatism. PGARA does not coalesce copies. Its cost changes only when the spill set changes.

Poletto and Sarkar replaced coloring with linear scan [7]. The allocator visits live ranges in one pass. The abstract reports that the algorithm is considerably faster than graph coloring and produces code almost as efficient. PGARA's greedy pass is also one packing pass, but it still builds an interference graph and does not allocate by interval endpoints.

Traub, Holloway, and Smith compared linear scan with graph coloring where compile time matters [8]. The retrieved abstract is truncated before any numeric table. Their comparison is compile time and generated-code performance, not a caller-supplied multi-profile estimate.

Wimmer and Mössenböck added lifetime holes and interval splitting to linear scan in the Java HotSpot client compiler [9]. The retrieved abstract does not include a numeric speedup. PGARA keeps one live range per virtual register. The range identifier is reserved for a later split that is not implemented.

Pereira and Palsberg modeled registers as a puzzle board, including precoloring and aliasing [10]. For PowerPC, x86, and StrongARM the puzzles are solved in polynomial time, with a heuristic for spilling. On SPEC CPU2000, the abstract reports compilation time as fast as the extended linear scan used by LLVM in that setting, and x86 code of similar quality. PGARA was not run on SPEC and does not model paired registers.

Burroughs used expected distance to the next use and placed spill instructions with block frequencies [11]. The allocator was evaluated against LLVM on SPEC CINT2000 for ARMv7. The abstract reports average reductions, versus LLVM, of more than 36% and 50% in dynamically executed stores and loads with static profiles, and 50% and 60% with dynamic profiles. Those percentages belong to that compari-

son. PGARA never emits spill instructions and never counts retired loads.

da Silva, Senefonte, and Attrot published an interference-graph dataset for machine-learning register allocation [13]. The abstract notes that coloring is NP-complete and that specialized training data are scarce. No accuracy figure was in the retrieved fragment. PGARA does not train a model and does not export their graph format.

Terci, Abdulhalik, Bayrakci, and Boz proposed BitEA, a bit-encoded evolutionary search [14]. The retrieved text does not state a spill-cost delta against a named baseline. PGARA's search is deterministic best improvement on one range. It stops at a local test rather than at a generation limit chosen for CPU time.

Oh, Jeong, Ro, and Yoon described CASH-RF, a hierarchical GPU register file with a small SRAM cache in front of STT-MRAM [15]. The method is a hardware and compiler co-design aimed at energy. PGARA names registers `p0`, `p1`, and so on. It has no cache hierarchy and no energy model.

Xiao, Chen, He, and Yang reallocated registers to loosen read-after-write dependencies on timing-speculative processors [17]. The method builds a dependency-aware interference graph. The retrieved abstract does not give a spill-operation count. A related paper uses that graph for voltage scaling [16]. A timing-error rate is not a PGARA cost.

Chen, Liu, and Liu studied allocation for application-specific processors in 5G micro-baseband kernels [18]. The method uses known data sizes and parallel access. PGARA accepts a textual three-address language with explicit block counts, not a baseband schedule.

Hu, Zhang, Wang, Liang, and Li allocated code that uses array-unit dual-usage register names [19]. The method keeps scalar and vector views of the same name consistent. PGARA's interference relation is pairwise presence at an instruction boundary, with no lane aliasing.

VenkataKeerthy *et al.* embedded multi-agent reinforcement learning in LLVM [12]. The preprint reports an interface for x86 and AArch64, and its abstract says the results match or outperform the LLVM allocators used in that study. This manuscript does not repeat a numeric speedup, and the tests here did not reproduce that comparison. RL4ReAl emits ISA-constrained code. PGARA returns an assignment map and an estimate, which the dashboard then draws.

Liu *et al.* surveyed profile-guided optimization in GCC and LLVM [1]. The preprint is a survey, not a new allocator. It identifies sampling overhead, changing workloads, and cross-architecture portability as open issues. PGARA does not collect profiles. It refuses to invent a missing block count.

Two further sources support the wording, not the comparison. Monderer and Shapley introduced potential games [21]. PGARA names its objective a potential and accepts only strict decreases. The source does not contain a proof that every unilateral payoff change equals the change in that function. Lattner and Adve presented LLVM [2]. PGARA is not an LLVM pass.

The manuscript does not reach thirty sources from 2018 through 2024. The brief showed an author-year reference layout. This draft uses numbered IEEE citations [20]. A 2024 GPU hierarchical-register paper was seen, but no usable abstract was retrieved, so it is not discussed. Unrelated search hits were omitted.

III. PROPOSED WORK

A. PROBLEM FORMULATION

The input is a program in a restricted three-address language, an integer register count k, and zero or more execution profiles. Instructions assign a constant, copy a register, or apply addition, subtraction, multiplication, or division. Terminators are `if` *register* `goto` *label*, unconditional jumps, and returns. A comparison such as `if t3 > 10 goto B1` is rejected. The false edge of a conditional branch is the next block in source order. The last block must end with a return or a jump. Each profile maps every block label to a nonnegative integer. Omitting a label is an error. The analyzer does not substitute zero, and the dashboard does not invent a count for a blank profile cell before a run.

Let R be the virtual registers that are live at any instruction boundary. One live range is created per such register. The allocation key is the range identifier, which equals the virtual-register name because ranges are not split. An assignment maps each range to p_i with $0 \le i < k$, or to spill. JSON `null` means spill. An assignment is illegal when two interfering ranges share a physical register. The integer k may be zero, in which case every range spills. The service accepts $0 \le k \le 256$. The dashboard slider runs from 1 to 16. Negative k is rejected. The objective is not a proof of minimum memory traffic. Kept registers contribute nothing to the estimate, so two legal packings of the same spill set have the same cost.

B. SYSTEM ARCHITECTURE

Figure 1 separates the library, the HTTP service, and the dashboard. Figure 2 shows the same path from the workspace controls. The library parses text or structured JSON, validates profiles, and runs the analyses in a fixed order. The service exposes health, a sample program, allocation, and the synthetic benchmark. The dashboard either shows a fixed fixture or calls the service. Sample mode does not compute an allocation. Backend mode does not substitute the fixture when a request fails.

The default service address is http://127.0.0.1:8741. The client reads `VITE_PGARA_API_URL` when that variable is set. Cross-origin requests are accepted from `localhost` and `127.0.0.1` on any port. Credentials are not enabled. Allocation requests send the robustness slider as the tradeoff, start the potential search from the all-spilled assignment, and use iteration budget 200. The synthetic suite posts an empty JSON object, so each built-in case keeps its own tradeoff and budget. Changing the editor, the slider, or the profile table sets a dirty flag and does not call the service until the user runs.

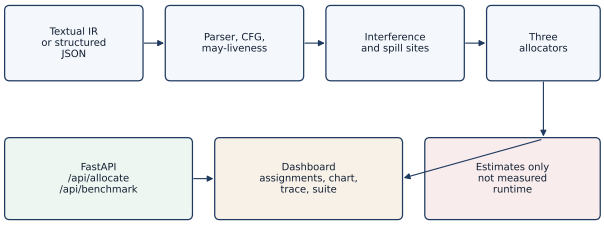


FIGURE 1. Library, service, and dashboard. Estimates are not measured program runtime.

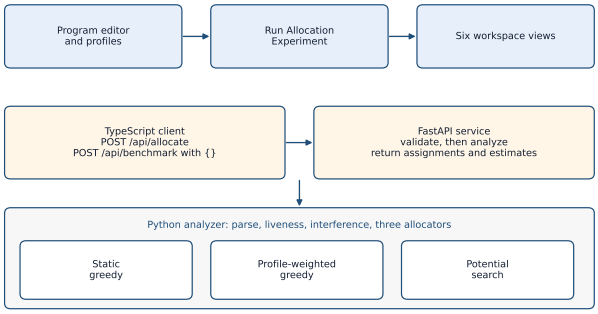


FIGURE 2. Workspace path. The run button sends allocation, then the synthetic suite with an empty body.

C. MODULES

The parser accepts one instruction per line, # comments, and block headers of the form `block name:`. Structured input uses opcodes `const`, `mov`, `add`, `sub`, `mul`, `div`, `br`, `jmp`, and `ret`. The caller must provide exactly one of source text or structured blocks. Limits are 32 profiles, 500 blocks, 5000 instructions, and 100000 source characters.

Control-flow construction records fall-through, the true branch, the false branch, and unconditional jumps. May-liveness iterates backward until block live-in and live-out stop changing. A use with no definition anywhere in the function is an undefined use. That check is not path-sensitive. A definition whose new value is never live is a dead definition and is not a store. The same virtual register used twice in one instruction contributes one use.

Interference is pairwise at instruction boundaries. The response includes nodes, edges, degrees, and per-instruction live sets. The dashboard draws the returned edges on a circle. That picture is a view of the graph, not a second analysis. The cost module is the only producer of estimated spill operations. Static greedy does not read profiles when it orders ranges. If profiles are present, the service still evaluates the static assignment under them. Profile-weighted greedy and the potential search require at least one profile. Requesting all three algorithms without a profile runs static greedy and marks the other two skipped.

The TypeScript client maps service names to display labels.

`static_greedy` is shown as Static greedy.
`profile_greedy` is shown as Profile-weighted.
`potential_game` is shown as Potential game.

The comparison chart reads every returned sum. It does not copy the algorithm selected in the assignment menu. A chart of zeros means every returned sum was zero. The client schema keeps assignments, spill lists, per-profile costs, the interference edges, and the potential-game decision trace. Per-instruction liveness remains available from the service response.

D. WORKSPACE

Table 1 lists the six routes. Overview shows configuration, the editor, the comparison chart, profiles, the decision trace, assignments, and the graph. The program editor keeps text and profile counts local until a run. Allocation lab repeats the chart and the assignment table next to the register slider. The interference view draws the circle layout for the selected algorithm. Benchmark comparison shows the chart, a per-profile chart, the assignment table, and the synthetic suite. The algorithm trace shows accepted potential-game moves in backend mode. In sample mode the trace is labeled as an illustration and is not produced by the search.

A metric strip shows the physical-register count, the number of live ranges, the lowest returned estimate, and allocator time. In sample mode those values stay on the fixture. In backend mode, allocator time is the recorded analyzer time for that result. It is not a spill penalty and is not used as a

TABLE 1. Dashboard routes. None of them allocates until the user runs an experiment.

Route	Role
`/`	Overview
`/program-editor`	IR text and profiles
`/allocation-lab`	Register budget and chart
`/interference-graph`	Returned edges
`/benchmark-comparison`	Suite and charts
`/algorithm-trace`	Potential-game moves

reported performance result in Section IV. Export writes a JSON file with a notice that states whether the payload is the fixture or an estimate.

E. WHAT THE CLIENT SENDS

Connection settings offer two sources. Sample dataset keeps the fixed illustration. FastAPI backend stores a URL, by default http://127.0.0.1:8741. Check connection calls `GET /api/health` and does not allocate. Apply stores the mode and the URL and does not allocate. Load sample, in backend mode, calls `GET /api/sample-program` and fills the editor. It does not run the three allocators.

Run Allocation Experiment is the only control that submits the editor. The allocation body carries the program text, the register count from the slider, one count per block for each profile, the selected algorithms, and the robustness slider as the tradeoff. When all three algorithms are selected, the client sends one request with algorithm `all`. The potential search is started from `all_spilled` with iteration budget 200. Load and store weights are omitted, so the service uses 1 and 1. After that response returns, the client posts an empty object to `POST /api/benchmark`. Each built-in case then keeps the tradeoff and budget in Table 2.

Each algorithm returns a profile row of estimated spill operations. The client adds those rows. That sum is the chart value. JSON `null` is drawn as a spill. A validation error stays on the page as an error banner. The sample fixture is not substituted. Edits made after a run set a dirty flag until the next run. The four numbers in the metric strip are the register count that was sent, the number of live ranges in the response, the lowest of the three sums, and the analyzer time attached to that lowest result. In sample mode the strip stays on the fixture: four registers, tradeoff 0.60, and the fixed spill totals 4200, 1800, and 1200.

F. FLOW

Figure 3 is the analyzer flow. Validation fails closed on unknown labels, negative counts, comparison branches, and extra JSON fields. Liveness is a fixpoint because loops require iteration. Only then do the allocators run. The dashboard default starts the potential search from all spilled, not from a greedy packing.

Figure 4 is the built-in tradeoff program. Spilling `a` costs 10 on both supplied profiles. Spilling `b` costs 19 on the profile named skew and 0 on quiet_b, with unit load and store weights. The instruction `b = b + 1` has an unused new value, so it contributes a load of `b` and not a store. Figure 5 shows that `a` and `b` interfere, so one register cannot keep both.

Figure 6 summarizes the potential search. The scan order is live-range identifier, then $p_0, p_1, \ldots$, then spill. An equal potential does not replace an earlier candidate. A zero iteration budget accepts no move. The no-improving-move reason is a local equilibrium of this neighborhood, not a proof of global optimality.

G. LIVENESS AND SPILL SITES

Block live-out is the union of successor live-in sets. Inside a block the transfer walks backward: a destination is removed, and every operand is added. Iteration stops when no block set changes, or reports an error if the count exceeds $2BN + 2$, where B is the number of blocks and N is the number of mentioned virtual registers, or 1 when none are mentioned. That cutoff is a guard, not a complexity theorem.

A destination that is live after its instruction is a store site. A destination that is not live is a dead definition. Each operand occurrence is a load site. If the backward walk disagrees with the block live-in, the analyzer raises an error instead of returning an assignment. The loop sample exists because a single backward pass would miss values that are live around the back edge. In that program the header says `if i goto body`, and the next block in source order is `exit`, so the false edge leaves the loop.

H. EQUATIONS

Costs are dimensionless estimated spill operations, not seconds and not joules. w_L and w_S are the integer load and store weights, each from 0 through 1000, with service defaults $w_L = w_S = 1$. $U(r)$ and $D(r)$ are the use sites and the live-definition sites of range r. Profile m supplies a nonnegative integer $f_m(b)$ for every block b. $M \geq 1$ whenever a profile-dependent quantity is evaluated.

$$C^{\text{static}}(r) = w_L|U(r)| + w_S|D(r)|. \tag{1}$$

$$C_m(r) = w_L \sum_{u \in U(r)} f_m(\text{block}(u)) + w_S \sum_{d \in D(r)} f_m(\text{block}(d)). \tag{2}$$

$$C_m(A) = \sum_{r : A(r) = \text{spill}} C_m(r). \tag{3}$$

Profile-weighted greedy orders ranges by a weight that does not depend on the assignment:

$$W(r) = \sum_{m=1}^{M} C_m(r). \tag{4}$$

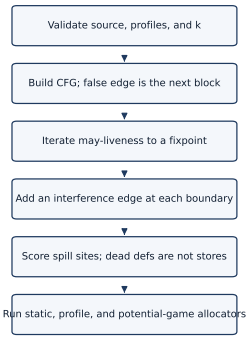


FIGURE 3. Allocation flow. A missing block count is an error.



FIGURE 4. Control-flow graph of the tradeoff program.

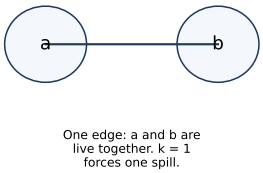


FIGURE 5. Interference graph of the tradeoff program. One register keeps exactly one of `a` or `b`.

Static greedy uses $C^{\text{static}}(r)$ instead. Ties break by ascending range identifier. With tradeoff $\lambda \in [0, 1]$, stored as a rational with denominator at most 10^6,

$$\Phi_\lambda(A) = (1-\lambda)\,\frac{1}{M}\sum_{m=1}^{M} C_m(A) + \lambda \max_m C_m(A). \tag{5}$$

A candidate A' is accepted only when

$$\Phi_\lambda(A') < \Phi_\lambda(A) \tag{6}$$

on those rationals. A physical register p is feasible for range r when no interfering neighbor already holds it:

$$\text{feasible}(r, p, A) \equiv \forall n \text{ adjacent to } r,\ A(n) \neq p. \tag{7}$$

An undirected edge exists when some instruction boundary holds both ranges:

$$\exists s \text{ such that } \{r, q\} \subseteq L_{\text{in}}(s) \cup L_{\text{out}}(s). \tag{8}$$

Equation (8) uses may-liveness, so an edge can exist for a pair that no single concrete path holds at once. If $w_L = w_S = 0$, every estimate is zero, the potential never strictly falls, and the search accepts no move from an all-spilled start. Greedy packing can still assign registers. That case is a unit test and is not in the tables below.

I. ALGORITHMS

Algorithm 1 is the shared packing procedure. It does not revisit an earlier spill if a later decision would have freed a register. The reported potential, when profiles exist, is computed after packing. Neither greedy variant claims to minimize that potential.

Algorithm 2 searches the neighborhood in (7). A cost that depends only on the spill set has no improving one-register move once a maximal greedy packing has filled every feasible register. Starting from greedy can therefore stop immediately. A unit test locks that behavior for the tradeoff program at $\lambda = 1$ when the search starts from profile-weighted greedy: it keeps `a` and spills `b`. The dashboard

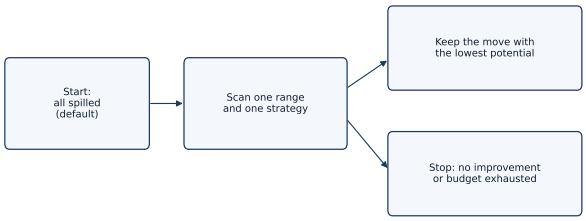


FIGURE 6. Potential search. Only a feasible one-range change that strictly lowers the potential is accepted.

Algorithm 1 One-pass greedy packing

Require: ranges R, interference edges, register count k, weight $w(r)$
Ensure: assignment A
  $P \leftarrow (\mathrm{p}_0, \ldots, \mathrm{p}_{k-1})$
  order R by decreasing $w(r)$, then by range identifier
  for each r in that order **do**
    $O \leftarrow$ physical registers held by already assigned neighbors of r
    if some register in P is absent from O **then**
      $A(r) \leftarrow$ the lowest such register
    else
      $A(r) \leftarrow$ spill
    end if
  end for
  Evaluate (3) and, if profiles exist, (5) on A

TABLE 2. Built-in programs. Counts are the values stored with each case.

Case	k	λ	Budget
sequential reuse	1	1/2	10
diamond	1	1/2	20
loop	1	1/2	20
profile vs. static	1	0	10
game average	1	0	10
game worst	1	1	10
zero registers	0	1/2	5
insufficient	1	1/2	20

does not use that start. Budget exhaustion is not described as optimality.

Table 2 records the eight built-in programs. An empty benchmark request uses those values. The robustness slider does not replace them.

The profile-versus-static program puts three updates of `a` in a block counted once and one update of `b` in a block counted 100 times, then returns `a`. The two game cases are the tradeoff program, one at $\lambda = 0$ and one at $\lambda = 1$. The profile skew counts blocks `def_a`, `def_b`, `use_a`, `use_b`, and `exit` as 2, 9, 3, 10, and 2. The profile quiet_b uses 2, 0, 3, 0, and 2. The equations, the two algorithms, and Figures 1–6 describe the source. They are not an approximation

Algorithm 2 Potential search from a valid start

Require: ranges, edges, k, profiles, λ, budget B, start mode
Ensure: assignment and stopping reason
  if start mode is all spilled **then**
    $A(r) \leftarrow$ spill for every range r
  else if start mode is static greedy **then**
    $A \leftarrow$ Algorithm 1 with (1)
  else
    $A \leftarrow$ Algorithm 1 with (4)
  end if
  $reason \leftarrow$ budget
  for B iterations, or none if $B = 0$ **do**
    $\phi \leftarrow \Phi_\lambda(A)$ using (5)
    $best \leftarrow$ none
    for each range r in identifier order **do**
      for each strategy in $\mathrm{p}_0, \ldots, \mathrm{p}_{k-1}$, then spill **do**
        if the strategy differs from $A(r)$ and satisfies (7) **then**
          Let A' be A with only r changed
          if $\Phi_\lambda(A') < \phi$ and $\Phi_\lambda(A')$ is the best so far **then**
            $best \leftarrow A'$
          end if
        end if
      end for
    end for
    if $best$ is none **then**
      $reason \leftarrow$ no improving move
      break
    end if
    $A \leftarrow best$ and record the move
  end for

theorem.

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. TOOLS AND SETUP

The analyzer is Python package version 0.1.0, checked with Python 3.12.3 [22]. HTTP models use Pydantic and FastAPI [23], [24]. Tests use pytest. From the `backend` directory, create a virtual environment, install `requirements.txt`,

TABLE 3. Parameters of the reported runs. Spill figures are dimensionless counts.

Parameter	Dashboard sample	Built-in suite
Register count k	2, 3, 4, 5, 6	Per case, including 0 and 1
Tradeoff λ	$3/5$	The case value
Load and store weights	1 and 1	1 and 1
Game start	All spilled	All spilled
Iteration budget	200	The case value
Profiles	Three supplied maps	The case maps

and run `python -m pytest`. The service is started with

```
python -m uvicorn pgara.api:app
-host 127.0.0.1 -port 8741
```

The numbers in Section IV-D come from the same `analyze` function the service calls, with load cost and store cost equal to 1 and the search started from all spilled. They are stored in `verified-results.json`. Wall-clock time was not saved.

The workspace is a TypeScript React application built with Vite [25], [26]. Routing uses TanStack Router. From the repository root, install dependencies and start the Vite development server. Open the printed URL. In Connection settings, choose the FastAPI backend, check health, and apply. Then run an experiment. Applying the connection does not allocate. No controlled timing study was run, and no CPU model is reported. The field `execution_time_ms` is checked only as a nonnegative value in a unit test. That check is not a performance result. No interface capture is inserted, because this draft does not include a screenshot taken from a running session.

B. PARAMETER SETUP

Table 3 records the runs. Load and store weights stay at the defaults because the dashboard does not expose them.

C. METRICS

Five quantities are supported. They are defined in Table 4. Estimated spill operations are plotted in Fig. 7 and listed in Tables 5, 6, and 7. Static spill operations and spilled-range identifiers are columns of those tables. Validity is the interference check after assignment. Spill-set agreement records whether the three algorithms returned the same spill set. Separate plots of those two checks would repeat a pass-or-fail observation, so they are not drawn. The dashboard's allocator-time tile is not a sixth metric.

D. OBSERVED RESULTS

Table 5 is the dashboard sample program at $\lambda = 3/5$. Source order is block B0, then B2, then B1, then B3. The false edge from B0 therefore enters B2. The profiles Hot loop, Balanced, and Cold-heavy are $(1000, 150, 850, 1000)$, $(1000, 500, 500, 1000)$, and $(1000, 800, 200, 1000)$ in that block order. At $k = 2$ every algorithm spills `c`, `d`, `t2`, and `t4`. The per-profile estimates are 5850, 5500, and 5200, so the sum is 16550 and the static count is 7. At $k = 3$ every algorithm spills only `d`, each profile contributes 1000, and the static count is 1. At $k = 4$, $k = 5$, and $k = 6$ every spill set is empty and every estimate is 0. At $k = 6$ the assignments still differ: static greedy places `t1` on p_0, `t3` on p_1, and `t5` on p_0, while the other two algorithms swap those three registers. Equation (3) does not see that difference. The three zeros on the comparison chart are three returned sums.

TABLE 4. Metrics the reported runs support. Units are counts unless noted.

Metric	What was computed
Estimated spill operations	Sum of (3) over profiles. Not seconds.
Static spill operations	Sum of (1) over spilled ranges.
Spilled live ranges	Identifiers with $A(r) = \text{spill}$.
Validity	Interference check. All tabulated runs passed.
Spill-set agreement	Whether the three spill sets were equal.

TABLE 5. Dashboard sample at $\lambda = 3/5$. All three algorithms share each row.

k	Sum	Static	Spilled ranges
2	16550	7	`c`, `d`, `t2`, `t4`
3	3000	1	`d`
4	0	0	none
5	0	0	none
6	0	0	none

Five built-in cases do not separate the algorithms. Table 6 lists them. They are validity and cost checks, not evidence that the search improves on greedy packing.

Three cases do separate a decision. They are plotted in Fig. 7 and listed in Table 7. For profile versus static, at $\lambda = 0$ and $k = 1$, static greedy spills `b` and the sum is 101. Profile-weighted greedy and the potential search spill `a` and the sum is 8. For game average, at $\lambda = 0$, all three spill `b` and the sum is 19. The search keeps `a` on p_0. For game worst, at $\lambda = 1$, static and profile-weighted greedy still spill `b` with sum 19. The search spills `a` and the sum is 20. The worst profile falls while the summed estimate rises. That is what (5) specifies at $\lambda = 1$. It is not a general win for the search.

In sample mode the displayed spill operations stay at 4200, 1800, and 1200 regardless of the editor. Those numbers are a fixture. They are not in Table 5. The zero-cost chart annotation and the empty benchmark request are implementation corrections. They do not create a new allocation result. Before the empty-body correction, a slider value of 0.6 would have run both game cases at $\lambda = 3/5$, and the search spilled `a` with sum 20 in both. The tables use each case's own tradeoff.

E. WHAT IS NOT COMPARED

No paper in Section II reports this estimate on this intermediate representation. Burroughs reports dynamic loads and stores against LLVM [11]. Pereira and Palsberg report compile time and code quality on SPEC [10]. The RL4ReAl

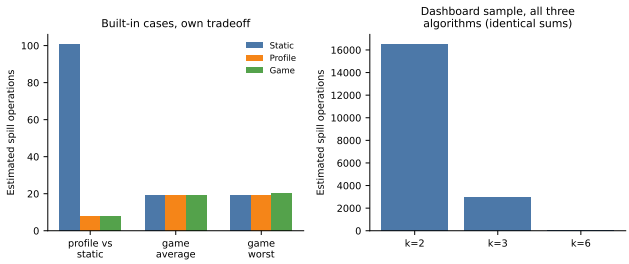


FIGURE 7. Left: estimated spill operations on the three separating cases. Right: the dashboard sample, where all three algorithms share each sum.

TABLE 6. Built-in cases with one shared spill set.

Case	Sum	Static	Spilled ranges
sequential reuse	0	0	none
diamond	3	3	`b`
loop	40	4	`s`
zero registers	6	6	`a`, `b`, `c`
insufficient	8	8	`b`, `c`, `d`, `f`

TABLE 7. Cases in which the spill decision is not identical.

Case	λ	Sum	Spill set
profile vs. static	0	101	static: `b`
		8	profile and game: `a`
game average	0	19	all three: `b`
game worst	1	19	static and profile: `b`
		20	game: `a`

preprint reports LLVM comparisons on x86 and AArch64 [12]. Those setups are not this experiment, so no quantitative comparison and no significance claim is made. There is no application suite, no LLVM or GCC comparison, no repeated timing protocol, and no sweep of weights or budgets. On the programs that were run, the search matches greedy except at $\lambda = 1$, where its summed estimate is 20 rather than 19.

V. CONCLUSION AND FUTURE WORK

PGARA keeps simultaneously live values in a finite file and estimates the rest from supplied block counts. Coloring, linear scan, puzzle solving, profile distance, evolutionary search, and learned allocators address that problem inside larger compilers, with metrics this repository does not compute. The prototype checks a three-address program, builds may-liveness and an interference graph, and packs ranges by a static count, a profile-weighted count, or a one-range potential search. FastAPI serves those results, and the React workspace draws them without replacing a failed call by the sample fixture.

The verified results are estimates on synthetic programs. On the dashboard sample, all three algorithms spill four ranges at two registers (sum 16550), one range at three registers (sum 3000), and none at four, five, or six registers. Profile weighting changes one built-in spill set from estimated cost 101 to 8. The potential search at $\lambda = 1$ improves the worst profile and raises the summed estimate from 19 to 20. Every tabulated assignment passed the interference check. None of these statements is a measured speedup.

The search is local, may-liveness is not path-sensitive, and ranges are not split or coalesced. Spill code is not inserted, so emitted loads are not counted. Profiles are trusted as given. Allocator time is recorded in the interface and was not evaluated as a performance metric.

A repeated timing protocol on a stated machine would make the recorded allocator time meaningful. A spill-code rewrite would allow a comparison of emitted loads and stores, which is the quantity Burroughs measured. Live-range splitting would use the identifier the data structures already keep. A study beyond the eight synthetic cases would show whether the $\lambda = 1$ choice is common. Until those measurements exist, the supported claim is the assignments and estimates reported above.

AUTHOR CONTRIBUTIONS

Not supplied. The author list and the contribution of each person must be added before submission. Contributors who do not meet the journal's authorship criteria belong in the acknowledgment.

ACKNOWLEDGMENT

Personal acknowledgments were not supplied. IEEE Access requires disclosure of generative artificial-intelligence assistance. This draft was prepared from the project repository with the assistance of a coding agent. The human authors must review every citation, number, and sentence and replace this paragraph with the disclosure they approve.

FUNDING

Funding information was not supplied. No grant number, sponsor, or supporting institution is claimed.

COMPETING INTERESTS

Competing interests were not supplied. No conflict is asserted or denied on behalf of an unnamed author.

DATA AND CODE AVAILABILITY

The analyzer, the tests, and the React workspace are in the project that contains this manuscript. Section IV uses

`verified-results.json`. The estimates can be recomputed from analyzer version 0.1.0 by the procedure in Section IV-A. A public archive and a DOI were not supplied. No external benchmark suite was downloaded for this draft.

REFERENCES

[1] B. Liu, Y. Huang, J. Gao, J. Shi, Y. Liu, Y. Sun, and W. Ji, "From profiling to optimization: Unveiling the profile guided optimization," arXiv:2507.16649, 2025.

[2] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis and transformation," in *International Symposium on Code Generation and Optimization*, 2004.

[3] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein, "Register allocation via coloring," *Computer Languages*, 1981.

[4] G. J. Chaitin, "Register allocation and spilling via graph coloring," in *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction*, 1982.

[5] P. Briggs, K. D. Cooper, and L. Torczon, "Improvements to graph coloring register allocation," *ACM Transactions on Programming Languages and Systems*, 1994.

[6] L. George and A. W. Appel, "Iterated register coalescing," *ACM Transactions on Programming Languages and Systems*, 1996.

[7] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Transactions on Programming Languages and Systems*, 1999.

[8] O. Traub, G. Holloway, and M. D. Smith, "Quality and speed in linear-scan register allocation," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1998.

[9] C. Wimmer and H. Mössenböck, "Optimized interval splitting in a linear scan register allocator," in *Proceedings of the 1st ACM/USENIX International Conference on Virtual Execution Environments*, 2005.

[10] F. M. Q. Pereira and J. Palsberg, "Register allocation by puzzle solving," in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2008.

[11] N. Burroughs, "Register allocation and spilling using the expected distance heuristic," *Software: Practice and Experience*, 2016.

[12] S. VenkataKeerthy, S. Jain, A. Kundu, R. Aggarwal, A. Cohen, and R. Upadrasta, "RL4ReAl: Reinforcement learning for register allocation," arXiv:2204.02013, 2022.

[13] P. Z. da Silva, H. C. d. M. Senefonte, and W. Attrot, "Interference graph dataset for machine learning-based register allocation," *IEEE Access*, 2024.

[14] G. S. Terci, E. Abdulhalik, A. A. Bayrakci, and B. Boz, "BitEA: BitVertex evolutionary algorithm to enhance performance for register allocation," *IEEE Access*, 2024.

[15] Y. Oh, I. Jeong, W. W. Ro, and M. K. Yoon, "CASH-RF: A compiler-assisted hierarchical register file in GPUs," *IEEE Embedded Systems Letters*, 2022.

[16] S. Xiao, J. He, X. Yang, H. Zhou, and Y. Yuan, "Timing error aware register allocation in TS," *Computer Systems Science and Engineering*, 2022.

[17] S. Xiao, Y. Chen, J. He, and X. Yang, "Read-write dependency aware register allocation," *Computer Systems Science and Engineering*, 2023.

[18] W. Chen, D. Liu, and S. Liu, "Register allocation compilation technique for ASIP in 5G micro base stations," *China Communications*, 2022.

[19] Y. Hu, X. Zhang, S. Wang, W. Liang, and K.-C. Li, "Research on global register allocation for code containing array-unit dual-usage register names," *Concurrency and Computation: Practice and Experience*, 2022.

[20] IEEE Access, "Preparing your article," https://ieeeaccess.ieee.org/authors/preparing-your-article/, 2026, accessed 13 Oct. 2026.

[21] D. Monderer and L. S. Shapley, "Potential games," *Games and Economic Behavior*, 1996.

[22] Python Software Foundation, "Python 3 documentation," https://docs.python.org/3/, 2026, accessed 13 Oct. 2026.

[23] Pydantic, "Pydantic documentation," https://docs.pydantic.dev/, 2026, accessed 13 Oct. 2026.

[24] FastAPI, "FastAPI documentation," https://fastapi.tiangolo.com/, 2026, accessed 13 Oct. 2026.

[25] React, "React documentation," https://react.dev/, 2026, accessed 13 Oct. 2026.

[26] Vite, "Vite documentation," https://vite.dev/, 2026, accessed 13 Oct. 2026.

• • •

AUTHOR NAME NOT SUPPLIED A biography was not supplied. Replace this paragraph with the corresponding author's biography. Do not add a photograph until one is provided.